

CIFAR-10 图像分类实验报告

基于 PyTorch 的 CNN 从零实现与 ResNet18 迁移学习对比

学生：罗诗琪 日期：2026 年 8 月

1. 实验背景与目标

本实验是暑期入门任务，目标是学习并掌握 Python 深度学习开发环境、PyTorch 框架以及常用科研实验工具，并在 CIFAR-10 数据集上完成图像分类实验。实验要求分别使用两种方法完成分类并对比结果：方法一为从零实现的卷积神经网络（CNN），方法二为基于 ImageNet 预训练权重的 ResNet18 迁移学习。通过两个实验，理解模型训练完整流程（数据如何进入模型、参数如何更新、如何评价结果），并练习 pandas、TensorBoard、matplotlib、scikit-learn 等常用科研工具。

2. 数据集介绍

CIFAR-10 是经典的图像分类基准数据集，共包含 60000 张 32x32 的彩色图片，分属 10 个类别：飞机、汽车、鸟、猫、鹿、狗、青蛙、马、船、卡车。其中 50000 张作为训练集，10000 张作为测试集。每张图片只有 32x32 像素，分辨率低、类内差异大（如猫和狗、鸟和飞机容易混淆），因此对模型的特征提取能力有一定要求，很适合入门实验。

3. 数据处理流程

数据加载由 dataset.py 完成：使用 torchvision 的 datasets.CIFAR10 自动下载并加载数据，通过 DataLoader 按批次送入模型。训练集与测试集的预处理不同：

训练集：随机水平翻转（数据增强）、四周填充 4 像素后随机裁剪回 32x32（增强平移鲁棒性）、转成张量、按 CIFAR-10 官方均值 (0.4914, 0.4822, 0.4465) 与标准差 (0.2470, 0.2435, 0.2616) 标准化。

测试集：只做转张量和标准化，不做任何随机增强，保证评估结果稳定、公平。

数据增强的目的：图片经过随机翻转和裁剪后，相当于在原始数据基础上生成了更多“视角”不同的样本，能有效抑制过拟合、提升模型泛化能力。标准化则把像素分布拉回均值 0、标准差

1，加速收敛并使训练稳定。训练时再从 45000 张训练集中切出 10%（5000 张）作为验证集，用于每轮监控过拟合；测试集只在全部训练结束后使用一次。DataLoader 参数：训练集 shuffle=True、测试集 shuffle=False、num_workers=2、drop_last=False。

4. 方法介绍

4.1 方法一：从零实现的 SimpleCNN

SimpleCNN 由 3 组"卷积块"和 1 个全连接分类层组成。每组卷积块依次为：卷积层（提取局部特征）、批归一化（稳定训练）、ReLU（引入非线性）、最大池化（下采样缩小尺寸）。输入 32x32 图片经过 3 次池化后变为 4x4，展平后送入全连接层输出 10 个类别的得分。具体结构如下：

层	输出形状	参数与作用
Conv2d(3, 32, 3, padding=1)	(32, 32, 32)	32 个 3x3 卷积核，提取颜色/边缘等底层特征，padding 保持尺寸不变
BatchNorm2d(32) + ReLU	(32, 32, 32)	归一化加速收敛；ReLU 提供非线性
MaxPool2d(2)	(32, 16, 16)	2x2 取最大值，尺寸减半，保留主要信息
Conv2d(32, 64, 3, padding=1)	(64, 16, 16)	64 个卷积核，提取组合特征
BatchNorm2d(64) + ReLU	(64, 16, 16)	同上
MaxPool2d(2)	(64, 8, 8)	下采样
Conv2d(64, 128, 3, padding=1)	(128, 8, 8)	128 个卷积核，提取高层语义特征
BatchNorm2d(128) + ReLU	(128, 8, 8)	同上
MaxPool2d(2)	(128, 4, 4)	下采样
Flatten	(2048,)	把 128x4x4 展平成 2048 维向量
Linear(2048, 10)	(10,)	映射为 10 个类别的得分

4.2 方法二：ResNet18 迁移学习

ResNet（残差网络）的核心思想是残差学习：让每个残差块学习"输入和期望输出之间的差值" $F(x)$ ，再与输入相加得到输出 $y = F(x) + x$ 。这种恒等映射式的捷径连接（shortcut）使梯度可以更顺畅地从深层传回浅层，解决了深层网络难以训练、性能反而退化的问题。

迁移学习：ResNet18 已经在 ImageNet（1000 类、上百万张图片）上完成预训练，学会了边缘、纹理、形状等通用的视觉特征。本实验直接加载预训练权重，只做三处结构修改以适应 CIFAR-10：

- 1) 第一个卷积层由 7x7、步长 2 改为 3x3、步长 1、padding 1，使 32x32 小图可以直接输入；
- 2) 去掉第一个卷积后的最大池化层（用 Identity 跳过），避免 32x32 输入被过度缩小；
- 3) 最后一层全连接由 1000 类改为 10 类。

由于特征提取部分继承了预训练参数，模型只需少量训练就能把"看图能力"迁移到 CIFAR-10 的 10 类分类上，因此通常收敛更快、精度更高。

5. 模型完整训练过程

5.1 训练流程

训练采用标准的监督学习流程，每个 epoch 把 45000 张训练图片完整过一遍，每批 128 张：前向传播得到预测得分；用损失函数计算预测与真实标签的差距；调用 `loss.backward()` 反向传播计算各参数梯度；调用 `optimizer.step()` 按梯度方向更新参数（步长由学习率控制）。每个 epoch 结束后在验证集上评估一次，并只在验证准确率创新高时保存模型。全部训练结束后，用测试集评估一次最终模型。

5.2 实验设置

项目	设置
损失函数	CrossEntropyLoss（交叉熵损失，分类任务的标准选择，内部包含 Softmax）
优化器	Adam，学习率 lr=1e-3（自适应学习率，收敛快、超参敏感度低）
批次大小	batch_size=128
训练轮数	方法一 CNN：30 轮；方法二 ResNet18：15 轮
验证集比例	训练集的 10%（5000 张），随机种子 42
数据增强	训练集：随机裁剪(填充4) + 随机水平翻转 + 标准化；测试集：仅标准化
硬件环境	NVIDIA GeForce RTX 5060 Laptop 8GB，CUDA 12.8，PyTorch 2.7.0+cu128

6. 实验结果

6.1 方法一：SimpleCNN

训练 30 轮，验证集最高准确率 80.50%（第 27 轮），测试集准确率 81.51%，测试集损失 0.5332。训练损失从 1.41 下降到 0.46，准确率从 49% 提升到 84%。

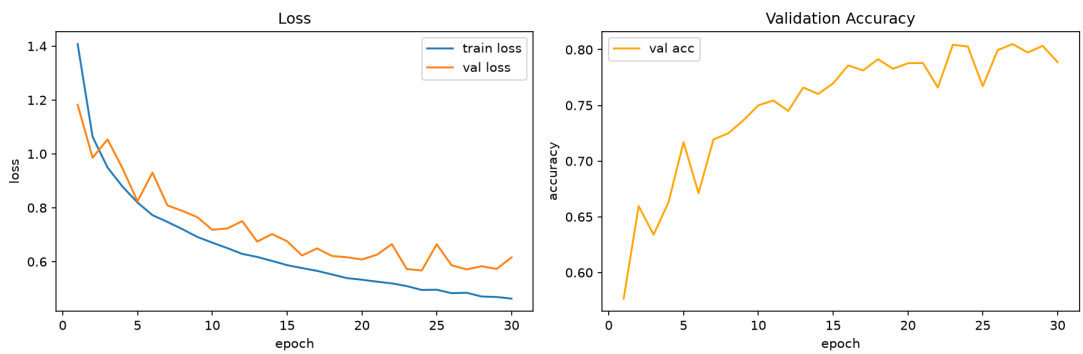


图 1 方法一训练/验证损失与验证准确率曲线

类别	精确率 Precision	召回率 Recall	F1
airplane（飞机）	0.8525	0.8150	0.8333
automobile（汽车）	0.9304	0.9090	0.9196
bird（鸟）	0.6450	0.8230	0.7232
cat（猫）	0.6061	0.7340	0.6640
deer（鹿）	0.8440	0.7520	0.7953
dog（狗）	0.8450	0.6270	0.7199
frog（青蛙）	0.8784	0.8520	0.8650
horse（马）	0.8862	0.8330	0.8588
ship（船）	0.8797	0.9140	0.8965
truck（卡车）	0.8938	0.8920	0.8929

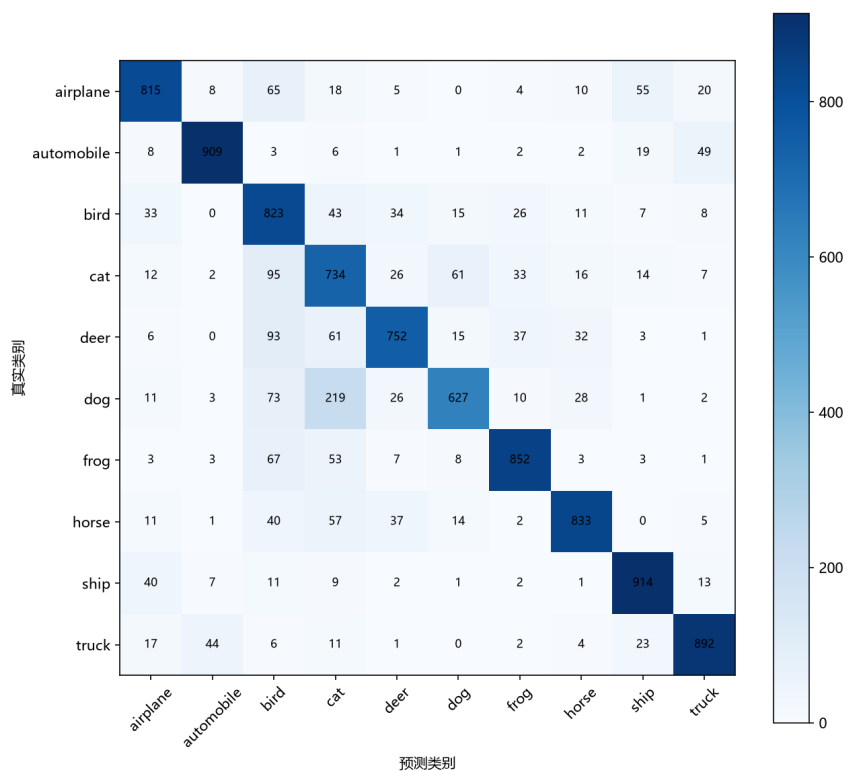


图 2 方法一测试集混淆矩阵

6.2 方法二：ResNet18 迁移学习

训练 15 轮，验证集最高准确率 91.52%（第 14 轮），测试集准确率 91.35%，测试集损失 0.2786。第 1 轮验证准确率即达 78%，收敛速度明显快于从零训练的 CNN。

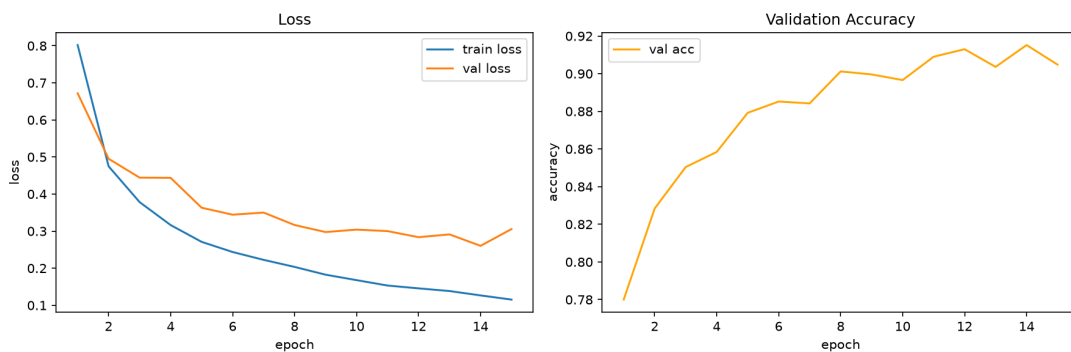


图 3 方法二训练/验证损失与验证准确率曲线

类别	精确率 Precision	召回率 Recall	F1
airplane (飞机)	0.9092	0.9510	0.9296
automobile (汽车)	0.9689	0.9360	0.9522
bird (鸟)	0.9327	0.8590	0.8943
cat (猫)	0.8500	0.7650	0.8053
deer (鹿)	0.9554	0.8990	0.9263
dog (狗)	0.8137	0.9000	0.8547
frog (青蛙)	0.8934	0.9640	0.9274

类别	精确率 Precision	召回率 Recall	F1
horse (马)	0.9224	0.9630	0.9423
ship (船)	0.9651	0.9400	0.9524
truck (卡车)	0.9365	0.9580	0.9471

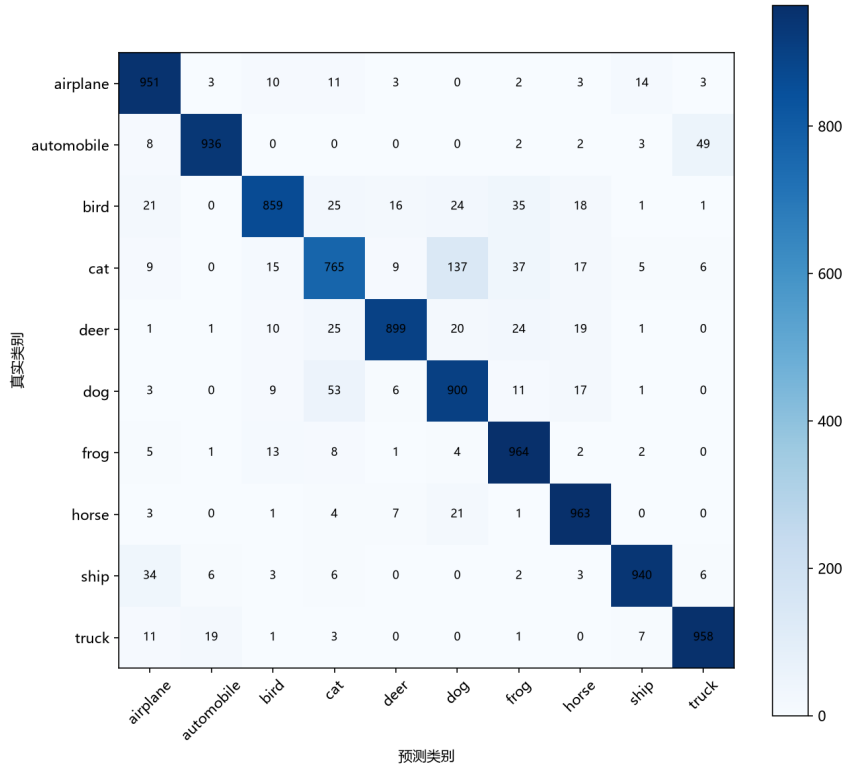


图 4 方法二测试集混淆矩阵

6.3 两种方法结果对比

指标	方法一 SimpleCNN	方法二 ResNet18
验证集最高准确率	80.50%	91.52%
测试集准确率	81.51%	91.35%
测试集损失	0.5332	0.2786
宏平均 F1	0.8168	0.9132
训练轮数	30	15
每轮耗时 (RTX 5060)	约 17 秒	约 38 秒

7. 模型性能分析

7.1 影响模型性能的因素

从本实验可以观察到几个影响性能的关键因素：(1) 数据增强——随机裁剪和翻转显著提升了泛化能力，若去掉增强，训练集准确率会很高但验证集明显下降（过拟合）；(2) 网络结构与容量——CNN 的卷积核数量和深度决定特征表达能力，本实验的简单 CNN 表达能力有限；(3)

初始化——随机初始化收敛慢且容易陷入较差的局部最优，而预训练初始化起点高、收敛快；(4)
超参数——学习率、批次大小、训练轮数直接影响收敛速度与最终精度。

7.2 未来改进方向

方法一可以从以下方向改进：加深网络（增加卷积块或改用更深的 CNN 结构）、使用 SGD + momentum + weight decay 配合余弦学习率调度、加入更多数据增强（如 Cutout、Mixup）、训练更多轮数等。方法二可以尝试：以更小的学习率微调全部参数、或只训练分类头冻结特征提取层、使用更强的预训练模型（ResNet50、EfficientNet 等）。此外，模型集成和标签平滑也能进一步提升精度。

7.3 两种方法性能差异及其原因

方法二在测试集上比方法一高约 9.8 个百分点（91.35% vs 81.51%），且只用了一半的训练轮数。原因主要有两点：一是 ResNet18 本身结构更深、具有残差连接，特征表达能力更强；二是更重要的是迁移学习——预训练权重已经在 ImageNet 上学会了通用的视觉特征（边缘、纹理、形状等），这些特征对 CIFAR-10 同样有效，模型只需在已有能力上做微调；而方法一的 CNN 所有参数都从随机初始化开始学，需要更多数据和更多轮次才能学到类似的特征，因此收敛慢、最终精度低。这一对比也说明：当目标数据量有限时，充分利用预训练模型是提升性能的有效手段。

8. 实验总结

本实验在 CIFAR-10 上完整走通了“数据处理、模型搭建、训练、验证、测试、结果分析”的深度学习全流程，分别用从零实现的 SimpleCNN 和 ImageNet 预训练的 ResNet18 完成了图像分类，测试集准确率分别为 81.51% 和 91.35%。通过对比实验理解了卷积网络的基本原理、残差结构与迁移学习的作用，并练习了 TensorBoard、matplotlib、scikit-learn 等工具。实验结果表明，在数据量有限的任务中，基于预训练权重的迁移学习相比从零训练具有明显优势。

附录：TensorBoard 可视化

在项目根目录运行 `tensorboard --logdir logs/tensorboard`，然后浏览器打开 `http://localhost:6006` 即可查看训练/验证损失与准确率曲线。